P0020r00 : Floating Point Atomic View

Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: hboehm@google.com
Author: JF Bastien
Contact: jfb@google.com
Author: James Reus
Contact: reus1@llnl.gov

Number: P0020
Version: 00
Date: 2015-09-23
URL: https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0020.rst
WG21: SG1 Concurrency

This paper proposes an extension to the atomic operations library [atomics] for atomic addition on an object conforming to the *atomic-view-concept* (see P0019, Atomic View) instantiated for a floating point type. A class conforming to the *atomic-view-concept* shall also provide the following operations when T is a floating point (FP) type. This capability is critical for high performance computing (HPC) applications. We leave open the question of whether the existing class **atomic<T>** should be similarly extended when T is a floating point type.

```
template<> struct atomic< FP > {
  FP fetch_add( FP operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
  FP fetch_add( FP operand , memory_order order = memory_order_seq_cst ) noexcept;
  FP fetch_sub( FP operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
  FP fetch_sub( FP operand , memory_order order = memory_order_seq_cst ) noexcept;
  void add( fp operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
  void add( fp operand , memory_order order = memory_order_seq_cst ) noexcept;
  void sub( fp operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
  void sub( fp operand , memory_order order = memory_order_seq_cst ) noexcept;
  void operator+=( FP operand ) volatile noexcept;
  void operator+=( FP operand ) noexcept;
  void operator-=( FP operand ) volatile noexcept;
  void operator-=( FP operand ) noexcept;
};

template<> struct atomic-view-concept < FP > {
  FP fetch_add( FP operand , memory_order order = memory_order_seq_cst ) const noexcept;
  FP fetch_sub( FP operand , memory_order order = memory_order_seq_cst ) const noexcept;
  void add( fp operand , memory_order order = memory_order_seq_cst ) const noexcept;
  void sub( fp operand , memory_order order = memory_order_seq_cst ) const noexcept;
  void operator+=( FP operand ) const noexcept;
  void operator-=( FP operand ) const noexcept;
};
```

***FP* fetch_add(*FP* operand , memory_order order = memory_order_seq_cst)**
***FP* fetch_sub(*FP* operand , memory_order order = memory_order_seq_cst)**
void add(*FP* operand , memory_order order = memory_order_seq_cst)
void sub(*FP* operand , memory_order order = memory_order_seq_cst)

Requires: The memory order shall not be memory_order_acquire, memory_order_acq_rel, or memory_order_consume.

Effects: Atomically replaces the value of the floating point object referenced by the atomic view with the sum of the previous value and operand for *add* or -operand for *sub*. Memory is affected according to the value of **order**. This is an atomic read-modify-write operation (1.10).

Returns: fetch_*op* Atomically, the value of the referenced object immediately before the effects.

Remark: The **add** and **sub** functions omit returning the value before the effects. Omission of a return value may allow a better performing implementation.

Remark: In the event of arithmetic overflow or other floating point exception, the behavior of this operation is implementation defined. It is a quality-of-implementation as to whether arithmetic overflow or other floating point exception is detectable in the resulting value; *e.g.*, a value of NaN.

void operator+=(*FP* **operand)**
void operator-=(*FP* **operand)**

Effects: += performs **add(operand)** and **-=** performs **sub(operand)**

Remark: These operators omit a return value to avoid conflicting with either the corresponding arithmetic operators for *FP* or the arithmetic operators for **atomic<integral>**. Note that the arithmetic operators for **atomic**<*integral*> are inconsistent with the corresponding arithmetic operators for *integral*.

```
int i(0);
( i += 1 ) += 2 ; // i+= returns an lvalue

atomic<int> ai(0);
( ai += 1 ) += 2 ; // error: i+= returns an rvalue
```